Personal Finance Manager (PFM) — Complete Technical Documentation

---

1. What Is PFM?

PFM is a full-stack **personal finance management web application** that helps individuals track their spending, set budgets, and understand where their money goes each month.

A user registers an account, logs their daily expenses under categories (Food, Transport, etc.), sets monthly spending limits per category, and gets automatic alerts when those limits are being approached. The dashboard gives an instant visual snapshot of daily, monthly, and yearly spending with pie charts and progress bars. Monthly reports can be exported as CSV for use in Excel or Google Sheets. Recurring expenses (like subscriptions or EMIs) can be configured once and auto-logged every month by a background scheduler.

---

2. Core Features

Feature	What It Does
Authentication	Register and login with JWT-secured sessions. No session cookies — fully stateless.
Dashboard	At-a-glance daily, monthly, yearly spending totals with pie chart category breakdowns and a monthly salary progress bar.
Expense Logging	Add, edit, delete expenses. Each expense has an amount, category, date, and optional note.
Search & Filter	Filter expenses by month, category, or keyword with debounced search.
Categories	System-wide default categories (Food, Transport, etc.) plus user-created custom categories with custom colours.
Budget	Set monthly salary and per-category spending limits. See real-time budget vs. actual comparisons with progress bars.
Spend Alerts	Automatic notifications when a category reaches 80%, 90%, or 100% of its limit, and when total spending crosses those thresholds of salary.
Recurring Expenses	Configure expenses that auto-log on a chosen day each month (e.g., Netflix on the 5th). Pause or delete at any time.
Monthly Reports	Visual monthly report with a day-by-day spending bar chart and category breakdown table. Export the full month as CSV.

---

3. Tech Stack

Backend

Layer	Technology
Language	Java 17

Framework	Spring Boot 3.2.5
Security	Spring Security 6 (stateless JWT)
ORM	Spring Data JPA / Hibernate
Database	MySQL 8
JWT Library	jjwt 0.11.5
Code Generation	Lombok
Build Tool	Maven
Scheduler	Spring `@Scheduled` (built-in)

Frontend

Layer	Technology
UI Library	React 18
Routing	React Router 7
Build Tool	Vite 5
HTTP Client	Axios (with interceptors)
Charts	Recharts (PieChart, BarChart)
Styling	Plain CSS with CSS variables, glassmorphism/dark theme

Infrastructure

Concern	Service
Backend Hosting	Render (Docker-based, free tier)
Frontend Hosting	Netlify (static SPA, free tier)
Database	MySQL (Render managed, or local)
Containerisation	Docker (multi-stage Maven → JRE Alpine image)
CI/CD	Netlify auto-deploy from GitHub; Render Docker deploy

---

4. Architecture Overview

```
┌──────────────────────────────────────────────────────────┐
│                     BROWSER (Netlify CDN)                 │
│                                                           │
│   React 18 SPA  ──  React Router  ──  Recharts / CSS       │
│          │                                                 │
│     Axios Client                                           │
```

```
│  (Bearer token injected on every request)                    │
└──────────────────────────┬──────────────────────────────────┘
                           │  HTTPS REST API
                           ▼
┌──────────────────────────────────────────────────────────────┐
│             Spring Boot 3 Backend  (Render)                  │
│                                                              │
│  JwtAuthenticationFilter  ──►  SecurityConfig               │
│         │                                                    │
│   Controllers (REST)                                         │
│      │                                                       │
│   Services (Business Logic)  ◄──  @Scheduled Scheduler      │
│      │                                                       │
│   Repositories (Spring Data JPA)                             │
│         │                                                    │
└─────────┼────────────────────────────────────────────────────┘
          │  JDBC
          ▼
┌────────────────┐
│   MySQL 8 DB   │
│   (Render / local)│
└────────────────┘
```

---

5. Database Schema

Seven tables are managed by Hibernate (`ddl-auto: update`).

users

Column	Type	Notes
id	BIGINT PK	Auto-increment
username	VARCHAR	Unique constraint
password	VARCHAR	BCrypt-hashed
role	VARCHAR	e.g. `ROLE_USER`

categories

Column	Type	Notes
id	BIGINT PK	
user_id	BIGINT	NULL = system default (shared), non-null = user-created
name	VARCHAR	e.g. "Food", "Transport"
color	VARCHAR	Hex colour e.g. `#f97316`
icon	VARCHAR	e.g. "food", "fuel"

is_default	BOOLEAN	True for seeded system categories

expenses

Column	Type	Notes
id	BIGINT PK	
user_id	BIGINT	FK to users
category_id	BIGINT	FK to categories
amount	DECIMAL	
note	VARCHAR	Optional free text
date	DATE	Defaults to today
created_at	DATETIME	Set on persist

Indexes: `(user_id, date)`, `(user_id, category_id)`

budget_profiles

Column	Type	Notes
id	BIGINT PK	
user_id	BIGINT	Unique — one profile per user
monthly_salary	DECIMAL	
currency	VARCHAR	`INR`, `USD`, `EUR`

category_budgets

Column	Type	Notes
id	BIGINT PK	
user_id	BIGINT	
category_id	BIGINT	Unique per (user_id, category_id)
limit_amount	DECIMAL	Monthly spending cap

recurring_expenses

Column	Type	Notes
id	BIGINT PK	
user_id	BIGINT	
category_id	BIGINT	

amount	DECIMAL	
description	VARCHAR	
day_of_month	INT	1–28
start_date	DATE	Auto-logging begins from this date
active	BOOLEAN	Pause/resume flag

Indexes: `(user_id)`, `(active)`

alerts

Column	Type	Notes
id	BIGINT PK	
user_id	BIGINT	
category_id	BIGINT	NULL = total budget alert
category_name	VARCHAR	Denormalised for display
threshold_percent	INT	80, 90, or 100
message	VARCHAR	Human-readable alert text
read	BOOLEAN	Default false
triggered_at	DATETIME	

Indexes: `(user_id, read)`, `(user_id, category_id, threshold_percent, triggered_at)`

---

6. Security Flow

Every HTTP request goes through this chain:

```
Request
  │
  ▼
JwtAuthenticationFilter
  ├── Path starts with /api/auth?  ──► Skip (public endpoint)
  │
  └── Extract "Authorization: Bearer <token>" header
        │
        ├── Token invalid / missing ──► Pass through (403 from Spring Security)
        │
        └── Token valid
              ├── Extract username from JWT claims
              ├── Load UserDetails from DB
              ├── Validate token against UserDetails
              └── Set Authentication in SecurityContextHolder
                        │
```

```
                    ▼
                Controller executes
```

JWT details:

- Algorithm: HMAC-SHA256
- Secret: configured via `JWT_SECRET` environment variable (min 32 chars)
- Expiry: 10 hours (36,000,000 ms), configurable via `JWT_EXPIRATION_MS`
- Token stored in browser `localStorage`, sent as `Bearer` header on every request
- 401 response → Axios interceptor clears token and redirects to `/login`

Password storage: BCrypt hashing via Spring Security's `BCryptPasswordEncoder`

CORS: Restricted to `localhost:5173`, `localhost:3000`, `pfm-q2kz.onrender.com`, and `*.netlify.app`

---

7. Complete API Reference

All endpoints except `/api/auth/**` require `Authorization: Bearer <token>`.

Authentication — /api/auth

Method	Endpoint	Body	Response
POST	`/api/auth/register`	`{ username, password }`	`200 OK` or `400 Username already taken`
POST	`/api/auth/login`	`{ username, password }`	`{ "token": "<jwt>" }`

Categories — /api/categories

Method	Endpoint	Notes
GET	`/api/categories`	Returns system defaults + user's custom categories
POST	`/api/categories`	Create a custom category `{ name, color, icon }`
PUT	`/api/categories/{id}`	Update name/color/icon (own categories only)
DELETE	`/api/categories/{id}`	Delete (own categories only)

Expenses — /api/expenses

Method	Endpoint	Notes
GET	`/api/expenses?month=2026-03`	Returns expenses for month; add `categoryId` or `search` to filter
POST	`/api/expenses`	`{ categoryId, amount, note, date }`
PUT	`/api/expenses/{id}`	Update any field
DELETE	`/api/expenses/{id}`	Own expenses only

Budget — /api/budget

Method	Endpoint	Notes

GET	`/api/budget/profile`	Get monthly salary and currency
PUT	`/api/budget/profile`	`{ monthlySalary, currency }`
GET	`/api/budget/categories`	List all category spending limits
PUT	`/api/budget/categories/{categoryId}?limit=5000`	Set or update a limit
DELETE	`/api/budget/categories/{categoryId}`	Remove a limit
GET	`/api/budget/comparison?month=2026-03`	Budget vs. actual for each limited category

Summary (Dashboard) — /api/summary

Method	Endpoint	Notes
GET	`/api/summary/daily?date=2026-03-20`	Daily total + category breakdown
GET	`/api/summary/monthly?month=2026-03`	Monthly total + category breakdown
GET	`/api/summary/yearly?year=2026`	Yearly total + category breakdown

Reports — /api/reports

Method	Endpoint	Notes
GET	`/api/reports/monthly?month=2026-03`	Full report with daily totals map, category breakdown, salary, savings
GET	`/api/reports/monthly/csv?month=2026-03`	Download CSV file (`Date,Category,Amount,Note`)

Recurring Expenses — /api/recurring

Method	Endpoint	Notes
GET	`/api/recurring`	List all recurring expenses
POST	`/api/recurring`	`{ categoryId, amount, description, dayOfMonth, startDate }`
PUT	`/api/recurring/{id}/toggle`	Pause or resume
DELETE	`/api/recurring/{id}`	Delete permanently

Alerts — /api/alerts

Method	Endpoint	Notes
GET	`/api/alerts`	All alerts, newest first
GET	`/api/alerts/unread`	Unread alerts only
GET	`/api/alerts/unread/count`	Count of unread alerts
PUT	`/api/alerts/{id}/read`	Mark one as read

PUT	`/api/alerts/read-all`	Mark all as read

---

8. Application Flow — End to End

New User Journey

```
1. User opens app  ──►  Redirected to /login
2. Clicks "Register"  ──►  POST /api/auth/register
3. Backend hashes password, saves user  ──►  "Registered"
4. User logs in  ──►  POST /api/auth/login
5. Backend returns JWT  ──►  Stored in localStorage
6. Axios interceptor injects token on all future requests
7. User lands on Dashboard
```

Logging an Expense

```
User fills form  ──►  POST /api/expenses
                           │
                      ExpenseService.create()
                           │
                      Saves to DB  (@Transactional)
                           │
                      AlertService.evaluate(userId)
                           │
                  Load category budgets + spending (2 queries)
                  Batch-load category names (1 query)
                  For each limit:
                    pct = spent / limit × 100
                    if pct ≥ 80/90/100 and not already fired
                    this month → Save Alert
                  Also check total spending vs. salary
                           │
                      Return saved expense DTO to frontend
```

Recurring Expense Scheduler

```
Every day at midnight (cron: 0 0 0 * * *)
  │
  ├── Load all active recurring expenses
  ├── Group by userId (avoids N+1 expense queries)
  │
  └── For each user:
        ├── Load all this month's expenses once
        │
        └── For each recurring rule:
              ├── Skip if startDate is in the future
              ├── Skip if today is before the target day this month
              ├── Check: any existing expense this month with same
              │   category, same amount, note starting "[recurring]"?
```

```
                └── If not found → create expense with note
                    "[recurring] <description>"
```

Alert Dedup Logic

Alerts fire at 80%, 90%, and 100% thresholds. Each `(userId, categoryId, thresholdPercent)` combination fires at most **once per calendar month** — checked via `triggered_at > first day of current month`.

---

9. Frontend Structure

```
frontend/src/
├── api/
│   ├── axiosClient.js      # Base client — injects token, handles 401
│   ├── auth.js             # login, register
│   ├── expenses.js
│   ├── categories.js
│   ├── budget.js
│   ├── summary.js
│   ├── reports.js
│   ├── recurring.js
│   └── alerts.js
│
├── components/
│   └── Loader.jsx          # Animated spinner used during API calls
│
├── pages/
│   ├── Dashboard.jsx       # Pie charts, monthly overview, salary tracker
│   ├── ExpensesPage.jsx    # Expense list with month nav, search, filter
│   ├── AddExpensePage.jsx  # Create / edit expense form
│   ├── CategoriesPage.jsx  # Manage categories
│   ├── BudgetPage.jsx      # Salary setup, limits, budget vs. actual
│   ├── RecurringPage.jsx   # Manage recurring expenses
│   ├── ReportPage.jsx      # Bar chart + table + CSV export
│   └── SettingsPage.jsx    # User settings
│
├── AuthPage.jsx            # Login / register toggle
├── Layout.jsx              # Sidebar (desktop) + top bar + bottom nav (mobile)
└── App.jsx                 # React Router route definitions
```

Page Routes

Path	Page	Auth Required
`/login`	AuthPage	No
`/`	Dashboard	Yes
`/expenses`	ExpensesPage	Yes
`/expenses/add`	AddExpensePage	Yes
`/categories`	CategoriesPage	Yes

`/budget`	BudgetPage	Yes
`/recurring`	RecurringPage	Yes
`/reports`	ReportPage	Yes
`/settings`	SettingsPage	Yes

Dev vs. Production URL Switching

```
// frontend/src/api/axiosClient.js
const axiosClient = axios.create({
  baseURL: import.meta.env.DEV
    ? 'http://localhost:9006'           // Vite dev server
    : 'https://pfm-q2kz.onrender.com'  // Production
});
```

`import.meta.env.DEV` is `true` during `npm run dev` and `false` in the production build.

---

10. Backend Package Structure

```
com.first.pfm
├── main/
│   └── StartApplication.java         # Entry point
├── config/
│   ├── SecurityConfig.java           # Filter chain, CORS, session policy
│   ├── JwtAuthenticationFilter.java  # Validates Bearer token per request
│   ├── JwtUtil.java                  # Token generate & validate
│   ├── MyUserDetailsService.java     # Loads user from DB
│   ├── SecurityUtils.java            # getCurrentUser() helper
│   └── DataSeeder.java               # Seeds default categories on startup
├── controller/                       # REST layer (one per domain)
├── service/                          # Business logic (one per domain)
├── repository/                       # Spring Data JPA interfaces
├── model/                            # JPA entities
├── dto/                              # Data transfer objects
└── exception/
    ├── GlobalExceptionHandler.java   # Maps exceptions to HTTP status codes
    └── ResourceNotFoundException.java
```

---

11. Configuration

application.yaml (base — all environments)

```yaml
spring:
  datasource:
    url: ${DB_URL:jdbc:mysql://localhost:3306/mydb}
    username: ${DB_USERNAME:root}
```

```yaml
    password: ${DB_PASSWORD:}
  jpa:
    hibernate:
      ddl-auto: update
jwt:
  secret: ${JWT_SECRET:...}
  expiration-ms: ${JWT_EXPIRATION_MS:36000000}
server:
  port: ${PORT:9006}
```

application-local.yaml (local development only)

```yaml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb?useSSL=false&allowPublicKeyRetrieval=true
    username: root
    password: <your_local_password>
server:
  port: 9006
```

Activated with `-Dspring.profiles.active=local` (pre-configured in `.vscode/launch.json`).

Production Environment Variables (Render)

Variable	Purpose
`DB_URL`	MySQL connection string
`DB_USERNAME`	Database username
`DB_PASSWORD`	Database password
`JWT_SECRET`	Min 32-character HMAC-SHA256 secret
`JWT_EXPIRATION_MS`	Token lifetime in milliseconds
`PORT`	Auto-set by Render

---

12. Deployment

Backend → Render

The Dockerfile performs a **multi-stage build**: Maven compiles the JAR in a full JDK image, then copies only the JAR into a lightweight JRE-Alpine image.

```dockerfile
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q
COPY src ./src
RUN mvn clean package -DskipTests -q
```

```dockerfile
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 10000
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Render detects the Dockerfile, builds and runs it as a Docker service. Environment variables are set in the Render dashboard.

Frontend → Netlify

`netlify.toml` at repo root:

```toml
[build]
  base    = "frontend"
  command = "npm run build"
  publish = "dist"
```

`frontend/public/_redirects` :

```
/* /index.html 200
```

This ensures React Router handles all client-side routes without Netlify returning a 404 on direct URL access. Deployments trigger automatically on GitHub push, or manually via `npx netlify-cli deploy --prod` .

---

13. Local Development Setup

Backend

```bash
# 1. Ensure MySQL is running with a database named "mydb"
# 2. Set credentials in src/main/resources/application-local.yaml
# 3. Run with local profile:
mvn spring-boot:run -Dspring-boot.run.jvmArguments="-Dspring.profiles.active=local"

# Or in VS Code: Run → "Run Spring Boot (local DB)"
# Starts at http://localhost:9006
```

Frontend

```bash
cd frontend
npm install
npm run dev
# Starts at http://localhost:5173
# Automatically targets http://localhost:9006 in DEV mode
```

---

14. Key Design Decisions

Stateless JWT over sessions — No server-side session storage. Every request is self-contained with the token. This makes horizontal scaling trivial and keeps the Render free-tier footprint minimal.

Hibernate `ddl-auto: update` — Schema is managed automatically from JPA entity annotations including `@Index`. No separate migration scripts are needed at this scale.

`@Transactional` on all write methods — Ensures atomicity. If saving an expense succeeds but a subsequent step fails, the expense save is still committed because alert evaluation runs after the save returns.

Batch category loading (N+1 elimination) — All service methods that return lists use `categoryRepository.findAllById(ids)` once and build an in-memory map, rather than querying per item. On a page with 30 expenses this reduces the query count from 31 to 2.

Single query for daily totals — The report page previously ran one `SUM` query per day of the month (28–31 queries). Now a single `GROUP BY DAY(date)` query returns all daily totals at once.

`import.meta.env.DEV` for URL switching — Vite's built-in flag removes the need for `.env` files just for the API base URL.

Recurring expense marker — Auto-logged expenses are prefixed with `[recurring]` in the note field so the dedup check can identify them without an additional FK column or separate table.

Search debouncing — The expense search input waits 350ms after the user stops typing before firing the API call, preventing a query on every keystroke.

Built with Spring Boot 3.2.5 · React 18 · MySQL 8 · Deployed on Render + Netlify